03강

프로그래밍 언어론

프로그래밍 언어 패러다임

컴퓨터과학과 김진욱교수

학습목차



1. 프로그래밍 패러다임
2. 프로그래밍 언어 패러다임
3. 여러 패러다임의 프로그램 예
4. 프로그래밍 언어와 프로그래밍 패러다임

01

프로그래밍 패러다임

1. 프로그래밍 패러다임

패러다임이란 무엇인가?

- 한 시대의 견해나 사고를 규정하는 양식, 규범, 체계 등을 통칭하는 말
- 토머스 쿤은 과학 분야의 발전에 따라 이전의 믿음과 체계가 흔들리고 새로운 체계로 대체되는 과정에 주목

패러다임 전환

- 천동설 → 지동설
- 저축이 미덕 → 소비가 미덕

1. 프로그래밍 패러다임

프로그래밍 패러다임

- 프로그램을 작성하는 전형적인 방식
- 특정 언어에 종속적인 것이 아닌
  프로그래머가 추구하는 프로그램 작성 방식

1. 프로그래밍 패러다임

프로그래밍 패러다임의 변화

- 주어진 데이터에 어떤 연산을 어떤 순서로 할지 결정하는 것이 중요
  - 명령형 프로그래밍 패러다임 등장

- 복잡한 데이터 처리를 위해 데이터 처리 방식 자체를 잘 정리하는 것이 중요
  - 절차형 프로그래밍 패러다임 등장

- 데이터가 저장된 메모리를 관리하는 것이 어려운 문제로 대두
  - 함수형 프로그래밍 패러다임 등장

1. 프로그래밍 패러다임

- 프로그래밍 패러다임의 변화(계속)
  - 계산 절차를 문제의 조건을 명시하는 규칙으로 생각
    - 논리 프로그래밍 패러다임 등장
    - 선언적 프로그래밍 패러다임의 한 종류
  - 같은 데이터에 대해 다른 처리 절차를 여러 개 명시해야 하는 경우가 흔히 발생
    - 객체지향 프로그래밍 패러다임 등장

1. 프로그래밍 패러다임

프로그래밍 패러다임 변화의 배경

응용 도메인의 변화

- 요구사항의 변화
- 계산 분야 → 다양한 응용 분야

프로그램 구성 방식의 변화

- 명령어 나열 → 존재하는 모듈의 조합

계산 모델의 변화

- 튜링기계 모델 → 새로운 방식의 계산 모델

02

프로그래밍 언어 패러다임

2. 프로그래밍 언어 패러다임

프로그래밍 패러다임 vs 프로그래밍 언어 패러다임

프로그래밍 패러다임

- 프로그램을 작성하는 방식이나 스타일

프로그래밍 언어 패러다임

- 해당 프로그래밍 언어가 지원하는 프로그래밍 패러다임
- 일반적으로 프로그래밍 언어는
  여러 프로그래밍 패러다임을 지원

2. 프로그래밍 언어 패러다임

주요 언어의 프로그래밍 패러다임

프로그래밍 패러다임 / 프로그래밍 언어	명령형	절차형	함수형	논리	객체지향
BASIC	○	○			
C	○	○			
C++	○	○	○		○
Haskell	○		○		○
Java	○	○	○		○
JavaScript	○	○	○		○
LISP		○	○		
Prolog		○		○	
Python	○	○	○		○
Smalltalk	○	○			○
Visual Basic	○	○			○

2. 프로그래밍 언어 패러다임

프로그래밍 패러다임의 양립성

- 새로운 패러다임의 등장으로 이전에 있던 패러다임이 더욱 공고히 완성됨
- 다양한 프로그래밍 패러다임은 서로 양립할 수 있음
- 주류 프로그래밍 패러다임이 바뀜에 따라 이를 언어가 수용하는 형태로 변화

예: Python

- 명령형 패러다임, 절차형 패러다임 지원

→ 함수형 패러다임, 객체지향 패러다임을 포함

03

여러 패러다임의 프로그램 예

3. 여러 패러다임의 프로그램 예

- 명령형 프로그래밍
  - 프로그램을 일련의 명령어 나열로 간주
  - 장점
    - 프로그램을 쉽게 이해할 수 있음
  - 단점
    - 프로그램이 복잡한 경우 효과적으로 다루지 못함

3. 여러 패러다임의 프로그램 예

명령형 프로그래밍 - BASIC

```
10 HOME
20 INPUT "Enter A: ";A : INPUT "Enter B: ";B
30 LET C = A - INT(A/B)*B
40 LET A = B : B = C
50 IF B > 0 GOTO 30
60 PRINT "GCD is ";A
70 END
```

```
Enter A: 54
Enter B: 96
GCD is 6
```

3. 여러 패러다임의 프로그램 예

- 절차형 프로그래밍
  - 프로그램을 서브루틴이라는 절차의 집합으로 구성하는 방법
  - 장점
    - 재귀호출을 사용하여 프로시저를 간단하게 정의

3. 여러 패러다임의 프로그램 예

절차형 프로그래밍 - Algol

```
PROC gcd = (INT a, b) INT: (
  IF b = 0 THEN
    a
  ELSE
    gcd(b, a MOD b)
  FI
);
main:(
  INT a = 54, b = 96;
  printf(($x“GCD("g“, "g“) is "gl$,a,b,gcd(a,b)))
)
```

```
GCD(+54, +96) is +6
```

3. 여러 패러다임의 프로그램 예

- 구조화 프로그래밍
  - goto 없이 프로그램을 작성하는 방법
  - 블록과 서브루틴을 이용
  - 구조화된 제어문을 사용
  - 현재 우리가 사용하는 대부분의 언어는 블록 구조 언어임

3. 여러 패러다임의 프로그램 예

구조화 프로그래밍 - C

```c
#include <stdio.h>

int main() {
    int a, b, c;

    printf("Enter a and b: ");
    scanf("%d %d", &a, &b);
    do {
        c = a % b;
        a = b;
        b = c;
    } while (b > 0);
    printf("gcd: %d\n", a);

    return 0;
}
```

```
Enter a and b: 54 96
gcd: 6
```

3. 여러 패러다임의 프로그램 예

객체지향 프로그래밍

- 프로그램은 서로 통신할 수 있는 객체의 집합
- 소프트웨어 재사용을 통해 빠른 개발을 지원함

객체(object)의 개념

- 상태를 유지하며 외부의 요청에 반응하는 데이터
- 상태 – 필드(멤버 변수), 반응 – 메소드(멤버 함수)
- 상태는 외부에 숨기고 메소드는 외부에 공개

3. 여러 패러다임의 프로그램 예

객체지향 프로그래밍 - Smalltalk

```
igcd := [ :a :b |
   |u v|
   u := a. v := b.
   [ v > 0 ]
     whileTrue: [ |c|
        c := u.
        u := v.
        v := c rem: v
     ].
   u abs
].

(igcd value: 54 value: 96) printNl.
```

```
6
```

3. 여러 패러다임의 프로그램 예

- 함수형 프로그래밍
  - 프로그램 수행이란 원하는 값을 구하는 것
    - 원하는 값은 초깃값에 함수를 반복적으로 적용함으로써 구할 수 있음
  - 데이터는 값으로, 명령어는 함수로 취급
    - 명령어가 데이터를 바꿀 수 없음
  - 대입문과 반복문 없음
  - 함수 자체도 값으로 취급 가능
    - 함수를 다루는 함수: 고계함수

3. 여러 패러다임의 프로그램 예

함수형 프로그래밍 - Haskell

```
module Main (main) where

main :: IO ()
main = putStrLn (show (igcd 54 96))

igcd a 0 = abs a
igcd a b = igcd (abs b) (a `mod` (abs b))
```

```
6
```

3. 여러 패러다임의 프로그램 예

- 선언적 프로그래밍과 논리 언어
  - 프로그램은 논리식 집합으로 구성
  - 논리식의 열거를 통한 선언적 프로그래밍 지원
  - 논리식은 명제나 술어로 나타냄
    - 술어는 인수를 받을 수 있음

3. 여러 패러다임의 프로그램 예

선언적 프로그래밍과 논리 언어 - Prolog

```
kitty(happy).

safe(happy).

pet(X) :- kitty(X), safe(X).
```

```
?- pet(X).
X = happy.
```

3. 여러 패러다임의 프로그램 예

선언적 프로그래밍과 논리 언어 - Prolog

```
gcd(U, 0, U).
gcd(U, V, X) :- V > 0, W is U mod V, gcd(V, W, X).
```

```
?- gcd(54, 96, GCD).
GCD = 6 .
```

04

프로그래밍 언어와 프로그래밍 패러다임

4. 프로그래밍 언어와 프로그래밍 패러다임

프로그래밍 패러다임의 특징

- 객체지향 프로그래밍 패러다임
  - 객체 사이의 통신을 통해 계산을 표현함

- 함수형 프로그래밍 패러다임
  - 함수의 적용을 통해 계산을 표현함

- 논리 프로그래밍 패러다임
  - 논리식의 진위를 증명하는 과정을 통해 계산을 표현함

정리하기

- 프로그램을 작성하는 전형적인 방식을 프로그래밍 패러다임이라고 함
- 프로그래밍 패러다임에는 명령형, 절차형, 함수형, 논리, 객체지향이 대표적임
- 프로그래밍 패러다임의 변화 배경으로는 응용 도메인의 변화, 프로그램 구성 방식의 변화, 계산 모델의 변화 등이 있음
- 하나의 프로그래밍 언어가 한 패러다임만 지원하는 것은 아님
- 다양한 프로그래밍 패러다임은 상호 보완적인 것으로 보는 것이 바람직함

04강

다음시간안내 ▶▶▶

구문론과 의미론